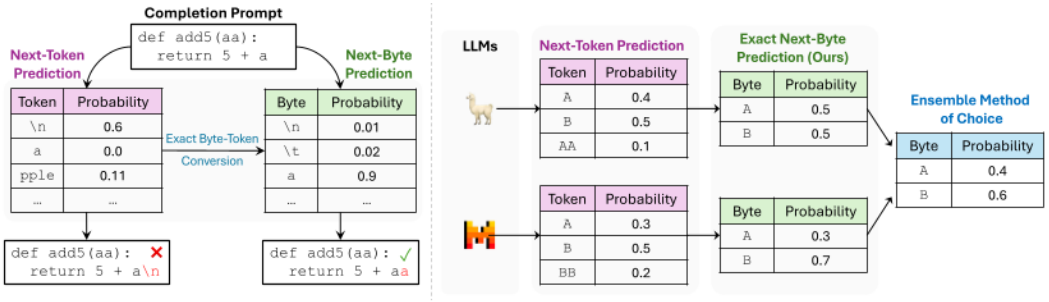


Figure 1: Left: when a prompt ends mid-token, as in this schematic code completion example, a tokenized LM suffers from tokenization bias: the correct continuation `a` (completing the argument name `aa`) receives zero next-token probability. Converting the same model's next-token predictions into exact next-byte predictions assigns it high probability. Right: mapping the next-token predictions of LMs with different vocabularies to statistically equivalent next-byte predictions (Section 1) puts them in a common space, where any ensemble method, such as averaging or a mixture of experts, can be applied.

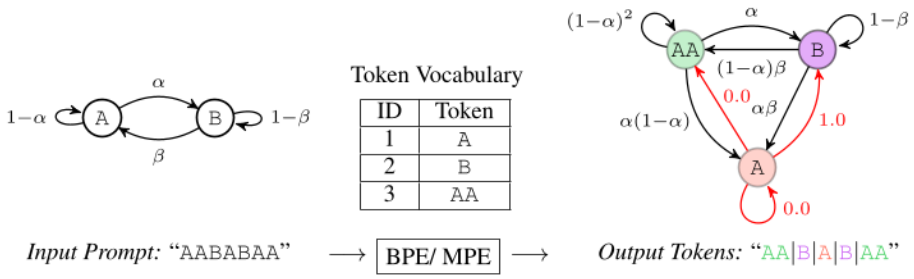


Figure 2: Tokenization bias on a first-order Markov chain over the bytes `A` and `B` (left), tokenized with the vocabulary {`A`, `B`, `AA`} by BPE or MPE (middle). Because two consecutive `A` bytes are always merged into the token `AA`, a standalone token `A` is always followed by `B` in encoded text. Given the context token `A`, the tokenized model therefore assigns probability 0 to the next token being `A` or `AA` and probability 1 to `B` (red edges, right), whereas in the original chain the byte after `A` is `A` with probability $1 - \alpha$. This occurs whenever a prompt ends with a token that is a prefix of another token, as is common in fill-in-the-middle (FIM) tasks.

EXHIBIT PREVIEW

Anonymous authors
Paper under double-blind review

1 METHODOLOGY

Definition 1 (Valid encoding)**.** *Placeholder.*

Algorithm 1 Cover encoding search (placeholder)

placeholder

Figures 1, 2, 3, 4, 5, 6; Tables 1, 2, 3.

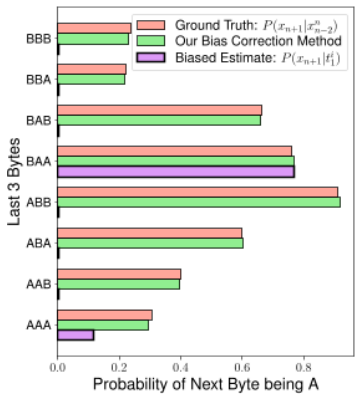


Figure 3: Tokenization bias and its correction on a third-order Markov chain. For each context of the last three bytes, bars give the probability that the next byte is `A`: ground truth $P(x_{n+1} \mid x_{n-2}^n)$ of the chain (red), the estimate obtained by conditioning a trained tokenized LM on its token history, $P(x_{n+1} \mid t_1^k)$ (purple), and the byte-level probability from our byte-token conversion applied to the same model (Section 1; green). The token-conditioned estimate is close to zero for six of the eight contexts, whereas the converted probabilities recover the ground truth.

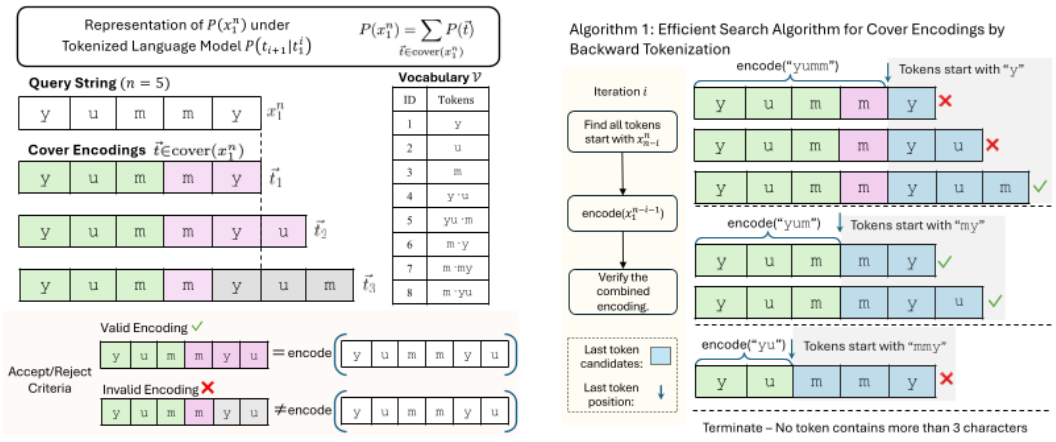


Figure 4: Left: the probability $P(x_1^n)$ of a byte string under a tokenized LM is the sum of the probabilities $P(\vec{t})$ of its cover encodings $\vec{t} \in \text{cover}(x_1^n)$, shown for the query string `yummy` ($n = 5$) and an eight-token vocabulary. An encoding is valid if re-encoding the bytes it decodes to returns the same tokens, and invalid otherwise (Definition 1). Right: Algorithm 1 finds the cover encodings by backward tokenization. At iteration i it lists the tokens that start with the last i bytes x_{n-i+1}^n, encodes the remaining prefix x_1^{n-i}, and keeps the combined encoding only if it is valid (green tick; red cross: invalid); the figure counts iterations from zero, so its labels x_{n-i}^n and x_1^{n-i-1} belong to iteration $i+1$ here. The search stops once the suffix is longer than every token (here, three characters), which is the loop bound of Algorithm 1. The example uses BPE.

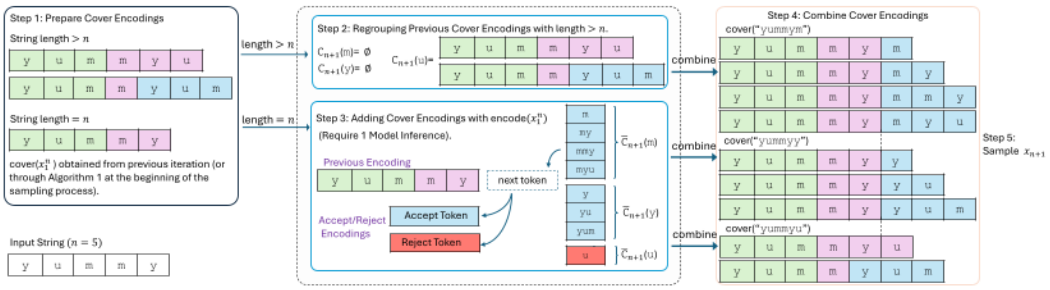


Figure 5: Sampling the next byte x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_1^k)$, continuing the example of Figure 4. Step 1: the cover encodings of x_1^n come from the previous iteration (or from Algorithm 1 at the start) and are split by the length of the string they decode to. Step 2: encodings longer than n bytes are regrouped by their $(n+1)$-th byte. Step 3: encodings of exactly n bytes are extended by next tokens predicted from $\text{encode}(x_1^n)$ (one model call), and each candidate is accepted or rejected by the validity criterion. Step 4: both groups are combined into the cover encodings of each candidate string $x_1^n x_{n+1}$. Step 5: x_{n+1} is sampled from the resulting byte-level distribution. The probabilities $P(\vec{t})$ are omitted for clarity.

Predictions	PSM Greedy	PSM @10	SPM Greedy	SPM @10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

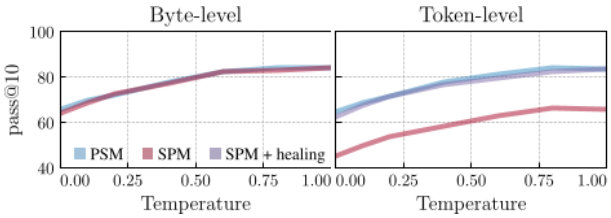


Figure 6: Pass rates (%) on the HumanEval Random Span infilling benchmark (Bavarian et al., 2022) with CodeLlama-7b, using the Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM) prompt templates. Left: greedy pass rate and pass@10, where @10 is the maximum over the temperature sweep on the right. The baselines token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) target prompts that end mid-token and are evaluated in SPM only (vertical bars). Right: pass@10 versus sampling temperature for byte-level and token-level predictions (the token-level panel also shows SPM with token healing). SPM forces predictions from the middle of a token: token-level greedy drops from 64.9 (PSM) to 45.0 (SPM), while byte-level predictions reach 63.9 in SPM, above both baselines, and are highest in every column.

Table 1: Accuracy (%) of general-purpose LMs with token-level and byte-level predictions on knowledge and reasoning benchmarks. Byte-level predictions are obtained from the same models without retraining. Voting and the byte-level ensemble combine the two best-performing models on each benchmark and are reported in the byte-level columns; token-level entries are not applicable (–). Voting was not reported for NQ, TriviaQA and GSM8K. Bold: best score per benchmark.

	MMLU		PIQA		NQ		TriviaQA		GSM8K	
Model	Token	Byte	Token	Byte	Token	Byte	Token	Byte	Token	Byte
Llama2-7B	45.4	45.7	78.1	78.2	25.0	23.4	58.4	58.1	15.1	12.5
Yi-1.5-6B	63.4	63.4	78.5	78.5	22.8	22.7	53.7	53.4	61.3	**61.5**
Mistral-7B-v0.3	62.1	62.1	80.1	80.3	28.5	28.8	63.6	63.5	39.0	39.0
Voting (top-2)	–	62.1	–	80.1	–	–	–	–	–	–
Top-2 Ensemble (ours)	–	**65.4**	–	**80.7**	–	**30.0**	–	**64.2**	–	55.8

Table 2: Pass@1 (%) of code LMs with token-level and byte-level predictions. The byte-level ensemble combines the two best-performing models on each benchmark (–: not applicable). Bold: best score per benchmark.

	HumanEval		MBPP	
Model	Token	Byte	Token	Byte
CodeLlama2-7b	32.3	28.7	40.6	42.4
CodeLlama2-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 Ensemble (ours)	–	**42.1**	–	**53.6**

Table 3: Effect of beam width on the byte-level FIM pass rate (%) on HumanEval. Width 1 matches the byte-level greedy results of Figure 6.

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	**67.7**	**66.8**

Table 4: Pass rates (%) on HumanEval Random Span infilling with CodeLlama-7b under the PSM and SPM prompt templates. Token healing and alignment are fixes for prompts ending mid-token and are evaluated in SPM mode only (–). Bold: best per column.

	PSM		SPM	
Method	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
+ token healing	–	–	62.7	83.8
+ token alignment	–	–	63.0	82.9
Byte-level (ours)	**65.7**	**84.1**	**63.9**	**84.3**

REFERENCES

Ben Athiwaratkun et al. Token alignment via character matching for subword completion. In *Findings of ACL*, 2024.

Mohammad Bavarian et al. Efficient training of language models to fill in the middle. In *arXiv*, 2022.

Gautier Dagan et al. Getting the most out of your tokenizer. In *ICML*, 2024.